

```

-- Questão 1
--a
CREATE OR REPLACE FUNCTION bloquear_exclusao()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF OLD.quantidade > 0 THEN
        RAISE EXCEPTION 'Ainda há exemplares. O livro (%) não será deletado do
banco.', OLD.titulo;
    END IF;
    RETURN OLD;
END;
$$;
--b
CREATE TRIGGER trg_bloquear_exclusao
BEFORE DELETE ON livro
FOR EACH ROW
EXECUTE FUNCTION bloquear_exclusao();

-- Questão 2
-- a
CREATE OR REPLACE FUNCTION log_exclusao_livro()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    insert into log_livro (titulo, quantidade_antiga, quantidade_nova,
data_log, mensagem)
values(OLD.titulo, OLD.quantidade_antiga, coalesce(OLD.quantidade_antiga -
1, 0), CURRENT_DATE, 'Livro removido.');
```

RETURN OLD;

```

END;
$$;

-- b
create trigger trg_log_exclusao
after delete on livro
for each row
execute function log_exclusao_livro();

-- Questão 3
--a
CREATE OR REPLACE FUNCTION validar_limite_estoque()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.quantidade > 100 THEN
        RAISE EXCEPTION 'Não é possível acrescentar mais unidades ao livro %.',
NEW.titulo;
    END IF;
    RETURN NEW;

END;
$$;

-- b
create trigger trg_validar_limite
before update on livro
for each row
```

```
execute function validar_limite_estoque();
```

-- Questão 4

--a) Triggers Before fazem validações antes de executar a ação, já a After faz a validação/registo após a ação. O que é bem apropriado para situação apresentada

-- Antes de um update é válido verificar se nenhuma regra de negócio será quebrada e após um update para guardar as informações alteradas logo após a ação.

-- Essas ações mantêm a integridade do banco e dados.

--

--b) A Before é a ideal para validações.

--c) A After garante que atualizações e registros automáticos ocorram após ações de update, delete por exemplo.

--d) Garantia de Integridade e Consistência: A ordem correta (Validação BEFORE -> Ação no Banco -> Auditoria AFTER) garante que lixo ou dados incorretos não entrem no banco e que o histórico reflita exatamente o que foi persistido. Fluxo de Dependência: Se o seu sistema depende de uma auditoria exata ou de bloqueios em cascata, inverter essa ordem pode gerar inconsistência de dados, logs fantasmas ou falhas de segurança.

-- Questão 5

--a) Se o banco de dados for acessado por mais de uma aplicação, todas as validações da aplicação são ignoradas. Isso abre brecha para inserção de dados "sujos" ou violações de regras de negócio. A redundância acaba, pois erros de implementação na aplicação podem sujar o banco.

--b) Uniformidade das regras de negócio independentes do serviço ou tecnologia que acessa ao banco. Proteção contra atividades maliciosas ou erros acidentais.

--c) Ajudam a garantir a ACID sem responsabilizar somente a aplicação das verificações de segurança, garantia de criação de rastreamento e bloqueia operações que infringem as regras de negócio.

--d) Auditoria de Folha de Pagamento ou Contabilidade: Em sistemas bancários ou de RH, sempre que o salário de um funcionário é alterado (UPDATE), uma trigger AFTER UPDATE pode registrar automaticamente em uma tabela de auditoria quem fez a alteração, a data, a hora, o valor antigo e o valor novo. Isso é fundamental para conformidade legal, auditorias externas e segurança da informação.

